Cover Page

Unit Code: COS30019
Unit Title: Introduction to Artificial Intelligence

Assignment number: Assignment 2a
Tutor: Dr. Afnan Ali
Lecturer: Dr. Afnan Ali

We declare that this is a group assignment and that no part of this submission has been copied from any other student's work or from any other source except where due acknowledgment is made explicitly in the text, nor has any part been written for us by another person.

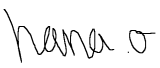
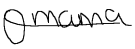
ID Number	Name	Signature
106194750	Hana Ouaida	
106203928	Omama Mohammad	
106196099	Asma Hossein	

Table of Contents

Table of Contents....2
Instructions.... 3
1. Introduction....3
1.1 Route Finding Problem.... 3
1.2 Graph and tree concepts....3
1.3 Algorithms Implemented.... 4
2. Features....4
2.1 Implemented Features.... 4
2.2 Not implemented....4
2.3 Known limitations and bugs.... 4
2.4 Notes for the marker.... 4
3. Testing.... 4
Test case 1: Multiple destinations.... 5
Test Case 2: Linear Chain....5
Test Case 3: Origin = goal....5
Test Case 4: Unreachable....6
Test Case 5: Cost v.s. Moves....6
Test Case 6: Triangle.... 7
Test Case 7: Cycle.... 7
Test Case 8: Equal Cost Paths.... 7
Test Case 9: Edges Unordered....8
Test Case 10: Disconnected.... 8
Test Case 11: Complete Graph.... 9
Test Case 12: Trap!....9
Test Case 13: Long Chain....9
Test Case 14: Grid.... 9
Test Case 15: Dead end....10
4. Insights.... 10
5. Research....10
5.1 Aim and method....10
CUS1: iterative-deepening DFS....11
CUS2: IDA*.... 11
The four other methods....11
5.2 Limitations of these results.... 11
Conclusion.... 11
7.Acknowledgements/Resources....12
8. References.... 12

Instructions

This solution was coded in Python, so the user must have the Python interpreter installed. If the user is on the Linux operating system, they may have to use 'python3' instead of 'python' to run the file (this was the case for one of our team members). However, it is generally assumed that the user is on Windows, as this is what was mentioned in the brief.

The correct usage from the command line is:
python search.py <filename> <method>

Please note that test files have been provided in a folder 'tests'. These are of the 'txt' file format. The test files have been modelled after the example file provided alongside the assignment brief. Some of these test files intentionally contain impossible problems. This is to test how the program handles such cases. The test files are descriptively named.

The methods available to use are:

- DFS
- BFS
- GBFS
- AS
- CUS1
- CUS2

These do not necessarily need to be capitalized when used.

The relative file paths of the test cases are:

- tests/t1_normal.txt
- tests/t2_linear_chain.txt
- tests/t3_origin_goal.txt
- tests/t4_unreachable.txt
- tests/t5_cost_vs_moves.txt
- tests/t6_oneway_detour.txt
- tests/t7_cycle.txt
- tests/t8_equal_cost_paths.txt
- tests/t9_edges_unordered.txt
- tests/t10_disconnected.txt
- tests/t11_complete_graph.txt
- tests/t12_trap.txt
- tests/t13_long_chain.txt
- tests/t14_grid.txt
- tests/t15_dead_end.txt

So, an example of a viable command might be:
python search.py tests/t7_cycle.txt cus1

When a goal is reached, the program outputs:
filename method
goal number_of_nodes (the destination node and the total number of nodes expanded to reach it)
path

1. Introduction

1.1 Route Finding Problem

Route Finding requires an agent to navigate between its starting point and its target destination by following a path through various network locations.

The assignment represents each location as a node which contains two-dimensional coordinates (x, y) on a flat surface and shows each road as a one-way edge which includes a cost value. Some roads can only be travelled in one direction. The agent begins its journey from an origin node but it needs to reach any destination node which will bring success.

The system needs to identify more than one target, but some of these targets might not be possible to reach. A good solution is a path with the lowest total cost.

The text file contains all problem information which includes node data and edge information together with starting points and target locations. Our program displays three pieces of information after each search because it shows the reached goal and search-tree node count and the complete path which led to the goal.

1.2 Graph and tree concepts

The map is a weighted directed graph. A path is a sequence of nodes joined by edges. Its cost is the sum of the edge costs, and its number of moves is the number of edges. Search builds a search tree: the root is the origin, and the children of a node are the graph nodes reachable by one edge. The same graph node can appear on several branches, but our program never creates a child already on its own path from the root, which prevents loops on two-way roads. The frontier holds nodes that are created but not yet expanded, and each algorithm differs in which one it expands next. The goal test is applied on expansion, and number_of_nodes counts every tree node created, including the root. Ties are broken by lower node number, then by earlier creation, as the brief specifies.
Uninformed algorithms use only the problem's structure. Informed algorithms also use a heuristic h(n), which for us is the straight-line distance from a node to the nearest destination.

2. Features

2.1 Implemented Features

- All six search methods: DFS, BFS, GBFS, A* (AS), CUS1 (iterative-deepening DFS) and CUS2 (IDA* on the number of moves).
- Command-line operation: `python search.py <filename> <method>`, run from the Windows command prompt. Method names are not case-sensitive.
- Output in the required format: `filename method`, then `goal number_of_nodes`, then the path.
- Cycle check: a child is never created if its node is already on its own branch, so two-way roads cannot cause infinite loops.
- Multiple destinations: the search stops at whichever destination it reaches first. The heuristic uses the nearest destination.
- Tolerant file parser: it accepts blank lines, extra spaces, Windows line endings, and destinations separated by `;` or `,`.
- Error handling: wrong arguments, a missing file, an unknown method, a badly formatted line, or a node with no coordinates each give a clear message and no crash.
- No recursion: all searches use loops and explicit stacks or queues, so deep graphs cannot hit Python's recursion limit.
- Test suite: 18 test files, each run with all six methods (108 runs) without crashes. See the Testing section.

2.2 Not implemented

- **Graphical user interface:** the program is command line only. The brief does not require a GUI.

2.3 Known limitations and bugs

- **CUS2 ignores path cost:** it finds the fewest moves, which is not always the cheapest path.
- **CUS1 and CUS2 node counts:** these are totals over all passes, so they are not directly comparable with single-pass methods.

2.4 Notes for the marker

- Tested on Windows with `cmd` and Python *[version 3.14.3]*.
- The method name for A* is `AS`, as in the brief.
- The test files are in the `tests` folder.

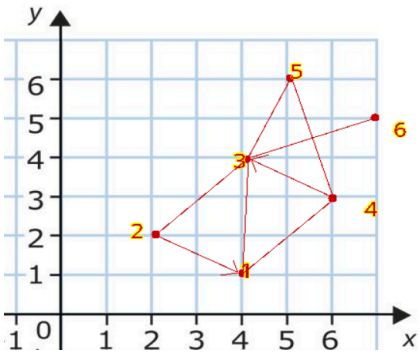
Method	Type	Chooses next	Finds
DFS	Uninformed	Deepest node	Any path
BFS	Uninformed	Shallowest node	Fewest moves
GBFS	informed	Lowest h(n)	A path quickly, not always the cheapest
A*(AS)	informed	Lowest g(n) + h(n)	Cheapest path, if h never over estimates
CUS1:iterative-deepening DFS	Uninformed	DFS repeated with depth limits 0, 1, 2	Fewest moves, with low memory
CUS2:IDA*	informed	DFS repeated with a rising limits on moves so far plus estimated moves left	Fewest moves, with low memory

3. Testing

For most of the tests, incomplete diagrams have been provided in this report. They do not contain information about the cost of each edge. This can instead be found in the test files. This decision was made to avoid visual clutter.

The results of every test for each algorithm are provided below, alongside brief discussion of what each case reveals about the program.

Test case 1: Multiple destinations



Origin: 2
Destinations: 5; 4

Costs are included in the test file. This is an example of a case with multiple destinations.

DFS
5 7
2 -> 1 -> 3 -> 5

BFS
4 10
2 -> 1 -> 4

GBFS
5 6
2 -> 3 -> 5

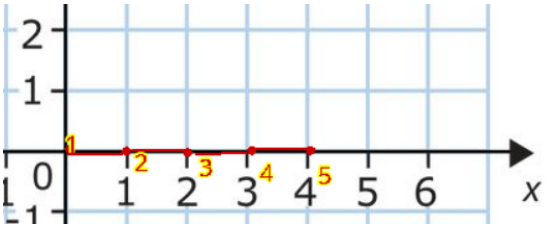
AS
4 8
2 -> 1 -> 4

CUS1
4 9
2 -> 1 -> 4

CUS2
4 5
2 -> 1 -> 4

What is interesting here is that most search algorithms reached the same destination using the same path, but they came to that solution by expanding a different number of nodes. For example, BFS expanded all of the nodes (10), but CUS2 only expanded 5 nodes.

Test Case 2: Linear Chain



Origin: 1
Destinations: 5

This is a very simple test. Every search algorithm should return the same result.

DFS
5 5
1 -> 2 -> 3 -> 4 -> 5

BFS
5 5
1 -> 2 -> 3 -> 4 -> 5

GBFS
5 5
1 -> 2 -> 3 -> 4 -> 5

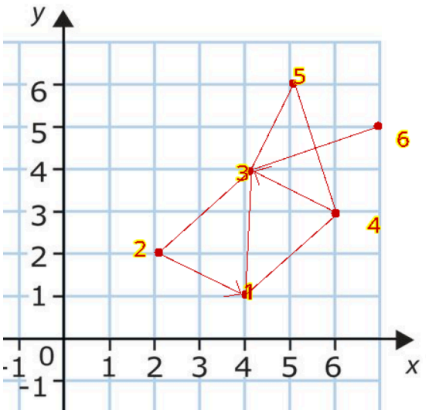
AS
5 5
1 -> 2 -> 3 -> 4 -> 5

CUS1
5 5
1 -> 2 -> 3 -> 4 -> 5

CUS2
5 5
1 -> 2 -> 3 -> 4 -> 5

All algorithms behaved as expected.

Test Case 3: Origin = goal



Origin: 3
Destinations: 3

In this test, the origin and the destination are the same node.

DFS
3 1
3

BFS
3 1
3

GBFS
3 1
3

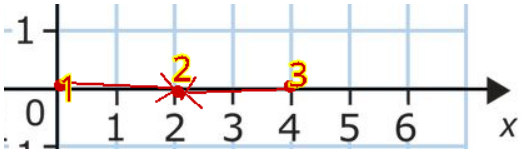
AS
3 1
3

CUS1
3 1
3

CUS2
3 1
3

Once again, all algorithms have returned the same result. This is exactly what we want.

Test Case 4: Unreachable



Origin: 1
Destinations: 3

This problem is intentionally impossible to solve. Here we are testing how the algorithms handle unreachable goals.

DFS
tests/t4_unreachable.txt DFS
No goal is reachable; 2

BFS
tests/t4_unreachable.txt BFS
No goal is reachable; 2

GBFS
tests/t4_unreachable.txt GBFS
No goal is reachable; 2

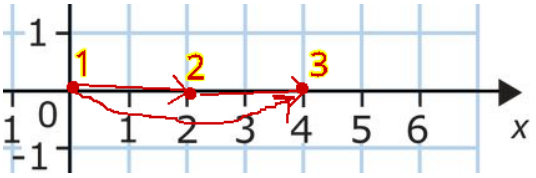
AS
tests/t4_unreachable.txt AS
No goal is reachable; 2

CUS1
tests/t4_unreachable.txt CUS1
No goal is reachable; 2

CUS2
tests/t4_unreachable.txt CUS2
No goal is reachable; 2

All algorithms behaved as they should. Wonderful!

Test Case 5: Cost v.s. Moves



Origin: 1
Destinations: 3

This case tests whether algorithms choose the less costly solution, or the solution with fewer moves.

DFS
3 4
1 -> 2 -> 3

BFS
3 4
1 -> 3

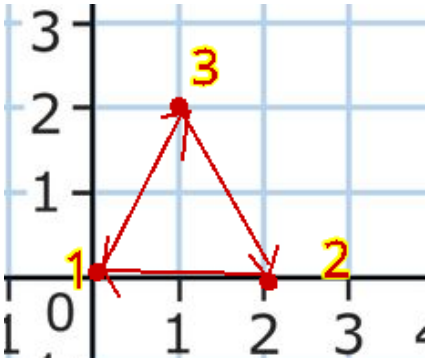
GBFS
3 3
1 -> 3

AS
3 4
1 -> 2 -> 3

CUS1
3 4
1 -> 3

CUS2
3 3
1 -> 3

Test Case 6: Triangle



Origin: 1
Destinations: 2

DFS
2 3
1 -> 3 -> 2

BFS
2 3
1 -> 3 -> 2

GBFS
2 3
1 -> 3 -> 2

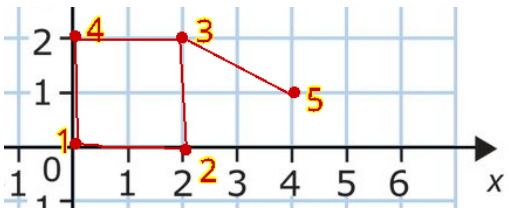
AS
2 3
1 -> 3 -> 2

CUS1
2 6
1 -> 3 -> 2

CUS2
2 5
1 -> 3 -> 2

As intended, all algorithms found the same path. However, some found it within 3 nodes, while CUS1 found it in 6. This is because of the iterative nature of CUS1.

Test Case 7: Cycle



Origin: 1
Destinations: 5

Here there are two paths to reach 5, both with the same cost.

DFS
5 6
1 -> 2 -> 3 -> 5

BFS
5 9
1 -> 2 -> 3 -> 5

GBFS
5 6
1 -> 2 -> 3 -> 5

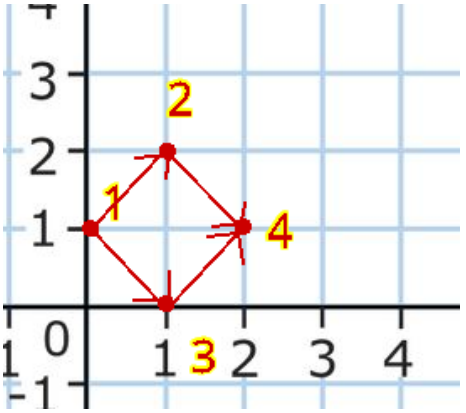
AS
5 9
1 -> 2 -> 3 -> 5

CUS1
5 15
1 -> 2 -> 3 -> 5

CUS2
5 10
1 -> 2 -> 3 -> 5

Much like the previous example, CUS1 took 15 nodes, while others took as little as 6! This test really showcases how iterative deepening makes CUS1 take the most computation power.

Test Case 8: Equal Cost Paths



Origin: 1
Destinations: 4

This tests our tiebreaker functionality. Both paths to reach the destination have the same cost.

DFS
4 4
1 -> 2 -> 4

BFS
4 5
1 -> 2 -> 4

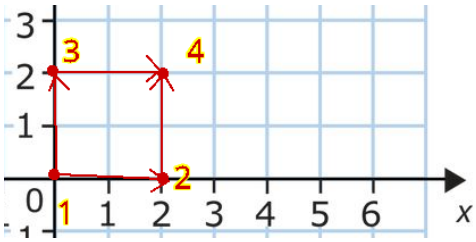
GBFS
4 4
1 -> 2 -> 4

AS
4 5
1 -> 2 -> 4

CUS1
4 8
1 -> 2 -> 4

CUS2
4 4
1 -> 2 -> 4

Test Case 9: Edges Unordered



In this test file, the edges were provided in a random order. This tests how the program handles edge order.

```
Edges:
(3,4): 2
(1,3): 2
(2,4): 2
(1,2): 2
```

DFS
4 4
1 -> 2 -> 4

BFS
4 5
1 -> 2 -> 4

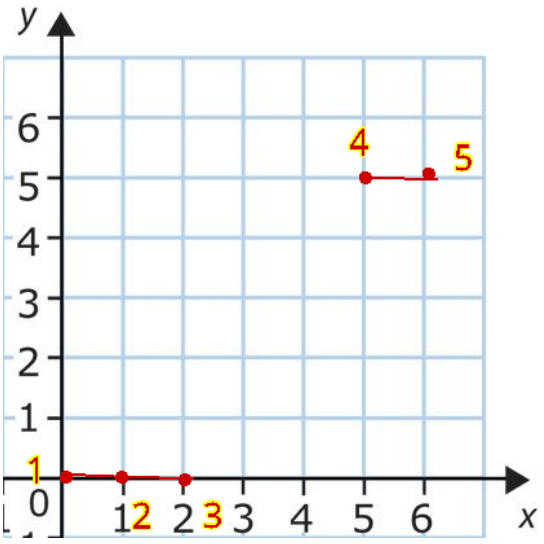
GBFS
4 4
1 -> 2 -> 4

AS
tests/t9_edges_unordered.txt AS
4 5
1 -> 2 -> 4

CUS1
tests/t9_edges_unordered.txt CUS1
4 8
1 -> 2 -> 4

CUS2
4 4
1 -> 2 -> 4

Test Case 10: Disconnected



Origin: 1
Destinations: 5

This case should be impossible, because the edges are disconnected.

DFS
No goal is reachable; 3

BFS
No goal is reachable; 3

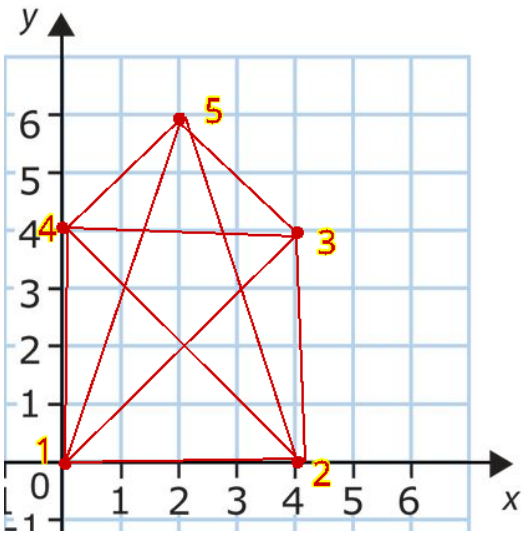
GBFS
No goal is reachable; 3

AS
No goal is reachable; 3

CUS1
No goal is reachable; 6

CUS2
No goal is reachable; 5

Test Case 11: Complete Graph



Here, every node is connected to every other node on the graph.
Origin: 1
Destinations: 5

DFS
5 11
1 -> 2 -> 3 -> 4 -> 5

BFS
5 14
1 -> 5

GBFS
5 5
1 -> 5

AS
5 8
1 -> 5

CUS1
5 6
1 -> 5

CUS2
5 5
1 -> 5

Test Case 12: Trap!

This case traps most of our algorithms! They take a path with cost 27 instead of one with cost 10. Note that A* is the only algorithm that found the cheaper path.

DFS
5 4
1 -> 2 -> 5

BFS
5 5
1 -> 2 -> 5

GBFS
5 4
1 -> 2 -> 5

AS
5 5
1 -> 3 -> 4 -> 5

CUS1
5 8
1 -> 2 -> 5

CUS2
5 4
1 -> 2 -> 5

Test Case 13: Long Chain

This test is similar to test 2, but with 20 nodes. We are making sure that the program can handle large trees.

All of the algorithms had the same output:

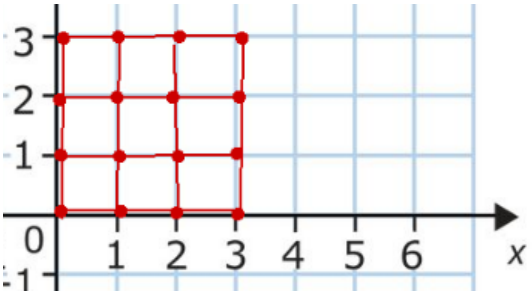
20 20
1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15 -> 16 -> 17 -> 18 -> 19 -> 20

Except for CUS1, which had this output!

20 210
1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 10 -> 11 -> 12 -> 13 -> 14 -> 15 -> 16 -> 17 -> 18 -> 19 -> 20

We can see that CUS1's weakness grows with larger trees.

Test Case 14: Grid



Another test with many nodes, but this one has even more connections.

DFS
16 22
1 -> 2 -> 3 -> 4 -> 8 -> 7 -> 6 -> 5 -> 9 -> 10 -> 11 -> 12 -> 16

BFS
16 175
1 -> 2 -> 3 -> 4 -> 8 -> 12 -> 16

GBFS
16 16
1 -> 2 -> 6 -> 7 -> 11 -> 12 -> 16

AS
16 111
1 -> 2 -> 6 -> 7 -> 11 -> 12 -> 16

CUS1
16 168
1 -> 2 -> 3 -> 4 -> 8 -> 12 -> 16

CUS2
16 25
1 -> 2 -> 3 -> 4 -> 8 -> 12 -> 16

Test Case 15: Dead end.



This is another impossible test. All algorithms return:
No goal is reachable; 1

4. Insights

Our results show that no single algorithm is best at everything. Each one trades path quality against nodes created and memory used.

Path quality: A* was the only method that found the cheapest path in every run. In Test 12 the other five all took the route costing 27, while A* found the one costing 10. The 6x6 grid with random costs gave the same picture: A* paid 32, BFS, CUS1 and CUS2 paid 55, GBFS paid 57 and DFS paid 159. BFS, CUS1 and CUS2 all aim for the fewest moves, so they only match A* when the cheapest path is also the shortest. Test 5 shows this clearly, because BFS, GBFS, CUS1 and CUS2 all chose the one-move road over the cheaper two-move route.

Nodes created: The informed methods created far fewer nodes. On the 6x6 grid with unit costs, BFS created 6,845 nodes and CUS1 created 5,614, while A* created 1,627 and CUS2 only 256. GBFS created just 28, but its path was not optimal. Test 14 followed the same pattern, with GBFS creating 16 nodes, CUS2 25, A* 111, CUS1 168 and BFS 175. A good heuristic lets the search ignore large parts of the tree, and the saving grows as the graph gets bigger.

Memory: BFS had to store its whole frontier, and its peak memory on the 6x6 grid was 608 KB. A* used 139 to 192 KB. The other four methods stayed between 2 and 7 KB. CUS1 and CUS2 use little memory because they only keep the current path, and CUS2 used the least of any method (2 KB) while still finding the fewest-move path.

The cost of iterative deepening: CUS1 repeats its upper levels on every pass, which hurts on long, narrow graphs. In Test 13 every other method created 20 nodes, but CUS1 created 210. On the grids the repeated work mattered much less, because the last level holds most of the nodes. CUS2 avoided most of this problem, since its f limit cuts off branches that cannot lead to the goal.

DFS: DFS often created very few nodes (4 in Test 12 and 22 in Test 14), but the paths it returned were unreliable. On the 6x6 grid it found a path of 30 moves when the best was 10. It was quick only because it did not look for a good path.

Which algorithm is better?

It depends on what the user needs. A* is the right choice for the cheapest path, as long as the heuristic never overestimates, which is true for straight-line distance. CUS2 is best when the fewest moves are wanted and memory is limited. BFS gives the same moves without a heuristic, but needs much more memory, and CUS1 is the low-memory alternative to BFS. GBFS suits cases where any reasonable path found quickly is enough. DFS is only suitable when path quality does not matter. All six methods gave the same results on the simple and impossible cases (Tests 2, 3, 4, 10 and 15), which gives us confidence that the shared code is correct.

5. Research

5.1 Aim and method

CUS1 and CUS2 had to be learned independently, so our research looked at how the two methods we chose, iterative-deepening DFS and IDA*, behave compared with the four standard methods. We ran all six methods on square grids from 3x3 to 6x6, with the origin in one corner and the single destination in the opposite corner. We tested two

versions of each grid: all roads cost 1, and random whole-number road costs from 1 to 9 (one fixed random seed). For each run we recorded the number of tree nodes created, the moves and cost of the path found, and the peak memory used. Run time was under 0.02 seconds in every case, so we don't report it.

CUS1: iterative-deepening DFS

We chose iterative-deepening DFS (IDDFS) because it is uninformed like BFS and finds the shallowest goal, but only needs to remember the current path. It runs depth-limited DFS with limits 0, 1, 2, … until a goal is found. We decided the search should stop with "no goal" when a pass is never cut off by the depth limit, because that means the whole tree has been explored. Its node count is the total over all passes.

On every grid it found a path with the same number of moves as BFS (10 on the 6x6 grid). Repeating the upper levels cost little extra, because the tree grows so fast that the last level dominates the total. On the 6x6 grid it created 5,614 nodes against BFS's 6,845, partly because it never creates the children of nodes at the depth limit. Its peak memory was about 3 KB against 608 KB for BFS.

CUS2: IDA*

We chose IDA* because it returns the same answer as A* on the same cost measure but, like IDDFS, keeps only the current path in memory. It runs a depth-first search that refuses to enter any node whose f value (moves so far plus estimated moves left) is above a threshold. The next threshold is the smallest f value that was refused.

Our main design decision was the heuristic. Straight-line distance is measured in distance units, not moves, so on its own it is not a valid estimate of moves. We divide it by the longest single edge in the graph and round up. No move can cover more than that distance, so the estimate never overestimates, and IDA* therefore finds the fewest moves. We count a refused child as created, because its f value had to be calculated.

IDA* found the fewest-move path on every grid. On the 6x6 grid with unit costs it created 256 nodes, against 1,627 for A* and 6,845 for BFS, using about 2 KB of memory. The trade-off is cost: CUS2 ignores road costs, so with random costs on the 6x6 grid its path costs 55 against 32 for A*.

The four other methods

- DFS is quick and uses little memory, but its paths are poor: 30 moves against the best 10 on the 6x6 grid, and 159 with random road costs.
- BFS always finds the fewest moves, but its memory use grows fastest (608 KB on the 6x6 grid).
- GBFS created the fewest nodes (28 on the 6x6 grid), but is not optimal. With random costs it paid 57 against 55 for BFS and 32 for A*.
- A* found the cheapest path in every run, with moderate memory (139 to 192 KB on the 6x6 grid).

Method	Moves in Path	Peak memory (KB)	Path Cost
DFS	30	7	159
BFS	10	608	55
GBFS	10	3	57
A*	10	192	32
CUS1	10	3	55
CUS2	10	2	55

5.2 Limitations of these results

The data comes from one grid layout, one random seed and one run per size, so the numbers show trends and not general rules. Peak memory was measured with Python's tracemalloc and can vary slightly between Python versions. Node counts are exact and repeat every time.

6. Conclusion

We implemented six tree-search methods for the route-finding problem: DFS, BFS, GBFS, A* and two custom methods, iterative-deepening DFS (CUS1) and IDA* (CUS2). All of them ran on the full test suite without crashing, and they handled the edge cases correctly, including a start that is also the goal, unreachable goals and disconnected graphs.

The results show a clear trade-off between path quality, nodes created and memory. A* found the cheapest path every time. CUS2 found the fewest-move paths using the least memory, and

GBFS created the fewest nodes but sometimes paid a higher cost. BFS was reliable but memory-hungry, DFS was cheap but gave poor paths, and CUS1 saved memory but repeated a lot of work on long, narrow graphs, as the 20-node chain showed.

The main limitation is that CUS2 counts moves and ignores road costs, so it can miss the cheapest route when costs vary. Our research results also come from one grid layout and one random seed, so they show trends and not general rules. With more time, we would make IDA* use path cost, test more graph shapes and seeds, and add a graphical display of the search. Overall, the best algorithm depends on what matters most: the cheapest path, the fewest moves, speed or memory.

7.Acknowledgements/Resources

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2), 100-107.

- The original paper introduced A search, which uses a heuristic to find least-cost paths efficiently and optimally.*

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. Artificial Intelligence, 27(1), 97-109.

- Introducediterative deepening, which combines the low memory use of depth-first search with the completeness and optimality of breadth-first search.

Python Software Foundation. (n.d.-a). collections - Container datatypes. Python 3 documentation.

- Documents the container types used in the implementation, such as `deque` for efficient queue operations in BFS.

Python Software Foundation. (n.d.-b). heapq - Heap queue algorithm. Python 3 documentation.

- Documents the heap-based priority queue used to pick the lowest-cost node in the informed searches (A and greedy best-first).*

Python Software Foundation. (n.d.-c). itertools - Functions creating iterators for efficient looping. Python 3 documentation.

- Documents the iterator tools, such as `count`, that support tie-breaking and efficient looping in the code.

Russell, S., & Norvig, P. (2021). Artificial intelligence: A modern approach (4th ed.). Pearson.

- The core AI textbook covering the search algorithms, completeness, optimality, and complexity analysis that the assignment is built on.

Anthropic. (2026). Claude [Large language model]

- Used to outline steps of the assignment

Swinburne University of Technology. (2026). COS30019 Introduction to AI: Assignment 2 Part A, Tree Based Search. Assignment 2A - Tree Based Search. Canvas.

- The assignment specification that defines the tree-based search task, input format, and required algorithms.

Poole, D. L., & Mackworth, A. K. (2023). Uninformed search strategies. In Artificial intelligence: Foundations of computational agents (3rd ed., Section 3.5). Cambridge University Press.

- Explained the uninformed strategies such as BFS, DFS, and iterative deepening, supporting the descriptions of those methods.

8. References

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence, 27*(1), 97–109. https://doi.org/10.1016/0004-3702(85)90084-0

Python Software Foundation. (n.d.-a). *collections — Container datatypes*. Python 3 documentation. https://docs.python.org/3/library/collections.html

Python Software Foundation. (n.d.-b). *heapq - Heap queue algorithm*. Python 3 documentation. https://docs.python.org/3/library/heapq.html

Python Software Foundation. (n.d.-c). *itertools - Functions creating iterators for efficient looping*.

Python 3 documentation. https://docs.python.org/3/library/itertools.html

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Anthropic. (2026). *Claude* [Large language model]. https://claude.ai

Swinburne University of Technology. (2026). *COS30019 Introduction to AI: Assignment 2 Part A, Tree Based Search* . Assignment 2A – Tree Based Search Canvas.

Poole, D. L., & Mackworth, A. K. (2023). Uninformed search strategies. In *Artificial intelligence: Foundations of computational agents* (3rd ed., Section 3.5). Cambridge University Press. https://artint.info/3e/html/ArtInt3e.Ch3.S5.html